ELEC-H-310
Th04: Synthesis of sequential circuits

Dragomir Milojevic
Université libre de Bruxelles

v3.3

Today

1. Sequential systems: classes & representation
2. State encoding
3. Complete example of a (very simple) sequential system
4. Optimisation of primitive state tables
5. D Flip-Flop

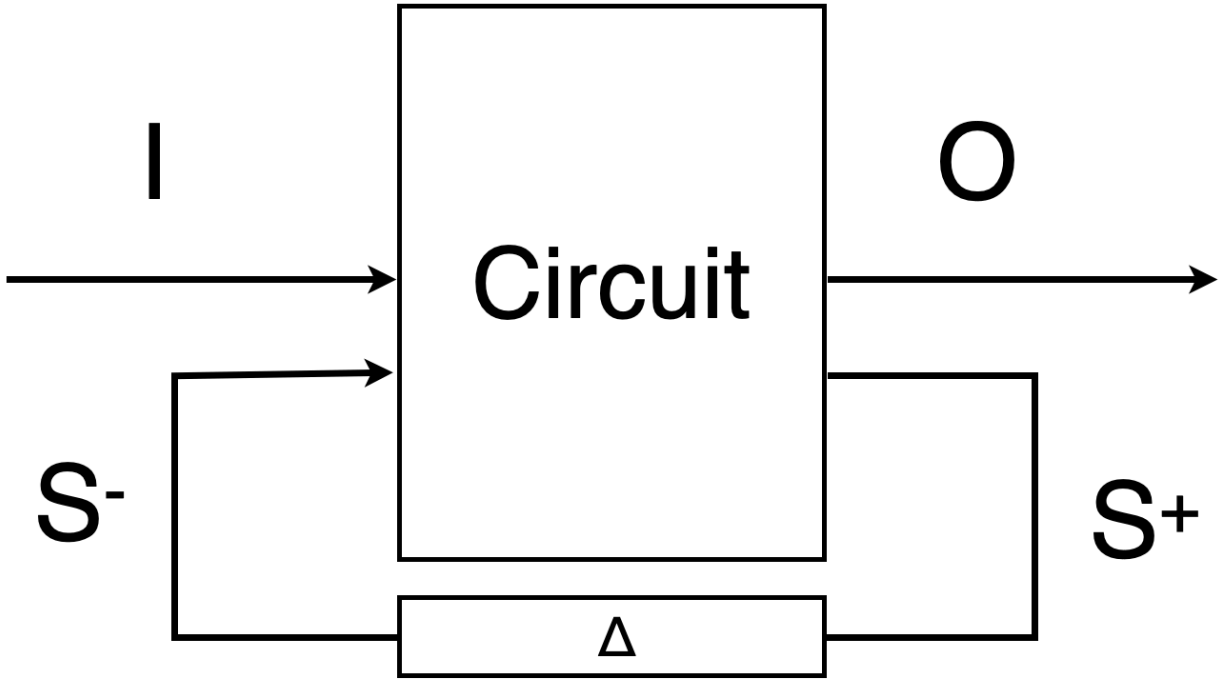
1. Sequential systems: classes & representation

Sequential systems – why and how?

- Combinatorial systems can't be used to control certain processes
- Last time we have seen the reservoir problem: level of the liquid in the tank can't be used alone to control the valve that will bring the liquid to the tank; in another words, we can't make a difference between the state of a "full tank being emptied" and/or "empty tank being filled"
- So, to compute the system output (i.e. to control the valve) **using the inputs only is not enough**; we need something more
- More is that sequential systems introduce the notion of system state, we have seen that we make a difference between current & future states
- Current system state is somehow "stored", "kept", "remembered" in the system – there is a notion of memory in sequential digital logic systems (more on this later...)

Memory: present & future

- State of the system is materialised using a feedback loop which allows us to make a difference between the present and the future
- Feedback loop introduces time difference between signals S^+ & S^-
- Time difference = delay in the wire, necessary to distinguish between y (present state) and Y (future state)



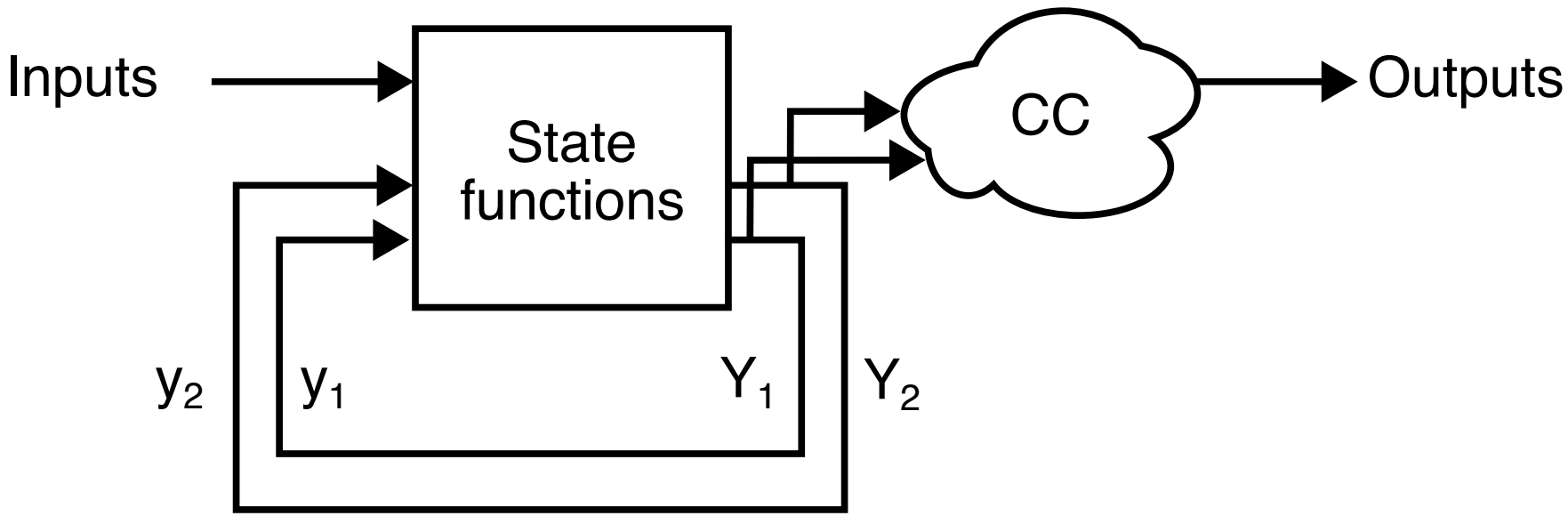
Delay Δ on the wire

States & sequential logic systems

- System state (internal state) is not necessarily seen by the observer (human or machine) outside of the system
- After all, what counts is what the systems does, or what it produces at the output
- In sequential systems the output is computed as a function of the current state AND **eventually** of the system inputs
- Here **eventually** mean that we have two different classes of sequential logic circuits:
  - Moore machine (not Gordon from Intel, but Edward Forrest Moore!)
  - Mealy machine
- Today let's just point the difference between the two & focus on Moore machine; we will see later what the Mealy machines are really capable off and why they are useful

Moore machines

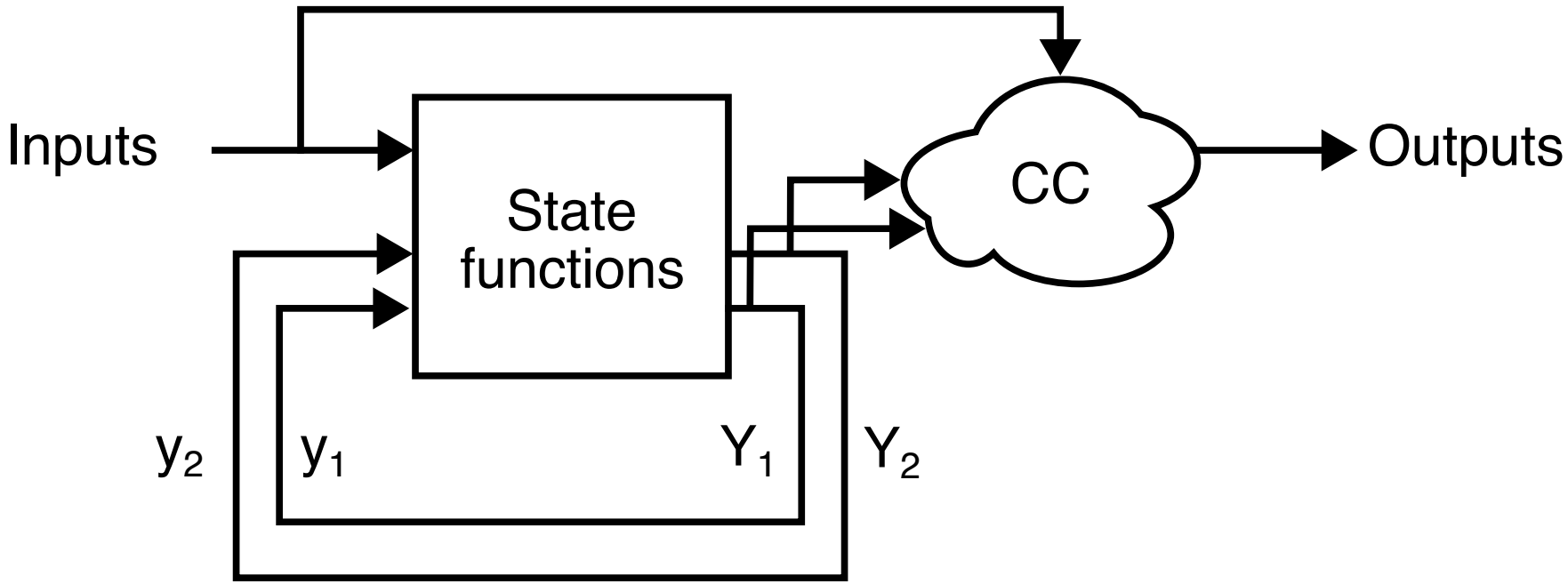
- Output (combinatorial) is function of state variables only



(we focus on this type of machines for a moment)

Mealy machines

- Output (combinatorial) is function of state variables and INPUTs



(we will see these machines in detail later on)

Sequential circuits – representation & synthesis

- Sequential systems can be represented using:
  - State graphs – for fun, nice to look at, but not that useful
  - State tables – we will used these extensively
  - Logic equations – crucial to enable actual circuit construction; we will use these & especially learn how to derive them from the state tables above
- All these provide a formal specification of the automata **evolution** in time (i.e. how the machines moves from one state to another based on inputs & current state)
- Note that the time here is seen more as an event, rather than some absolute value ... real circuits will though have to deal with actual time
- One can easily switch from one to another representation (you only need one formal representation, that must be correct) – easy part to switch from one mode to another
- Hard part is to derive initial system specification from verbal specs to capture correct system behaviour (doable, but you need to practice)

2. State encoding

State encoding

s^+ \ ab	00	01	11	10	Z
1	**1**	2	**1**	–	0
2	1	**2**	3	–	0
3	–	2	**3**	4	1
4	1	–	–	**4**	1

s^-

- State table showed uses decimal numbers to represent states, but it could be anything: letters, symbols, etc. as long as the predicted future exists as present (any future must have present)
- Logic circuits use binary alphabet: $B \in \{0, 1, -\}$
- To enable digital logic implementation state table must be encoded in binary $\rightarrow$ state encoding
  - ▹ Encoding of n states – done with $log_2 n$ state variables of 1 bit

State encoding example

- System has 4 states & we should attribute **unique binary code** to each state; we need **two state variables**
- Two binary state variables produce exactly 4 states
- Let's pick an arbitrary code first (restrictions will come ...), so:

`1→00; 2→01; 3→11; 4→10`

				ab	
S^+	00	01	11	10	Z
1	**1**	2	**1**	–	0
2	1	**2**	3	–	0
3	–	2	**3**	4	1
4	1	–	–	**4**	1

S^-

				ab	
S^+	00	01	11	10	Z
00	**00**	01	00	–	0
01	00	**01**	11	–	0
11	–	01	**11**	10	1
10	00	–	–	**10**	1

S^-

Encoded state table

- Encoded state table: each bit of a code is a separate, dedicated logic function that will become a dedicated feedback loop
- In our example: 4 states, 2 state variables (y_2,y_1), 2 logic functions, 2 different circuits with 2 different feedback loops
- Note the difference between y_2,y_1 & Y_2, Y_1 (lower & upper case)

	ab				
Y_2, Y_1	00	01	11	10	Z
00	**00**	01	00	–	0
01	00	**01**	11	–	0
11	–	01	**11**	10	1
10	00	–	–	**10**	1

y_2,y_1

State table is like a crystal ball!
Give it a present & inputs and it will tell you the future

Encoded state table → K-Maps

Y_2, Y_1 \ ab	00	01	11	10	Z
00	**00**	01	00	–	0
01	00	**01**	11	–	0
11	–	01	**11**	10	1
10	00	–	–	**10**	1

y_2, y_1

y_2y_1 \ ab	00	01	11	10
00	0	0	0	-
01	0	0	1	-
11	-	0	1	1
10	0	-	-	1

$$Y_2 = ab' + y_1 a$$

y_2y_1 \ ab	00	01	11	10
00	0	1	0	-
01	0	1	1	-
11	-	1	1	0
10	0	-	-	0

$$Y_1 = a'b + y_1 b$$

3. Complete example of a (very simple) sequential system

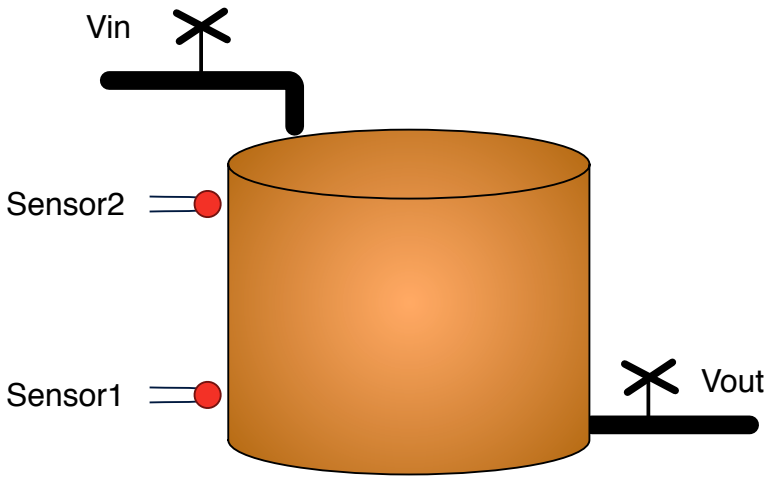
Starting point – optimised state table

For the example of the reservoir:

1. Write the state table
2. Draw the state graph
3. Encode the states
4. Derive K-Maps corresponding to feedback logic functions
5. Derive the output (new)
6. Enumerate all logic functions
7. Draw the circuit diagram

1. State table 1/3

- We suppose following initial state: reservoir is full and thus introduce state `1` for which we assume that inputs $ab = 11$
- Important! other initial situations are possible too: e.g., empty reservoir, and we open V_{in} until we reach `Sensor2` etc.
- From `1`, tank could be emptied only: new state `2` for $ab = 01$
- From state `2`, tank could be eventually re-filled (rain?), transition back to state `1` with $ab = 11$
- Output Z that controls V_{in} is '0' for both `1` & `2`

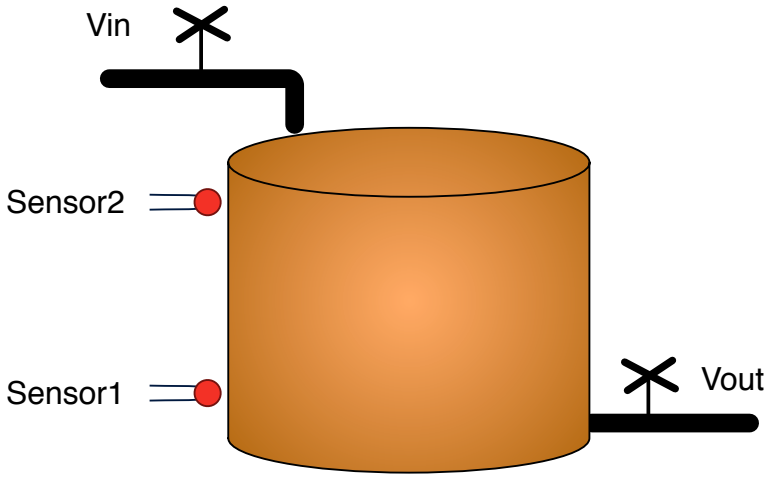


s^+ \ ab	00	01	11	10	Z
1	?	2	**1**	?	0
2	?	**2**	1	?	0
3	?	?	?	?	?
4	?	?	?	?	?

s^-

1. State table 2/3

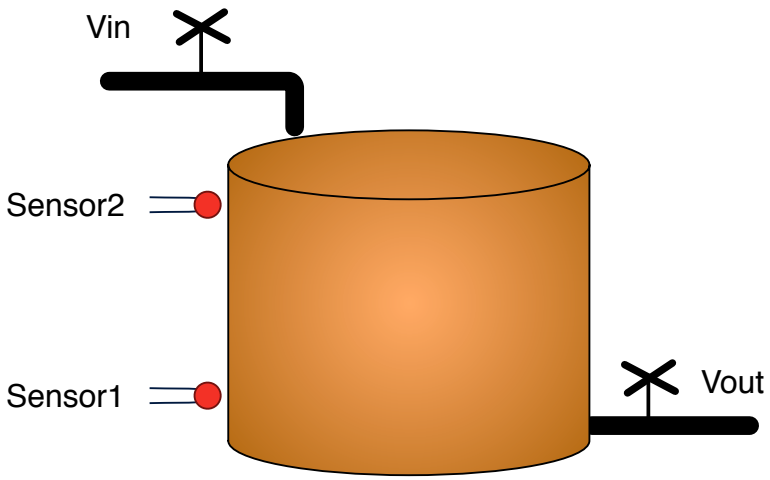
- From **2**, if tank continues to get emptied, it will reach `Sensor1`, so when ab=00 $\rightarrow$ new state **3**
- Output should be set to '1' since we need to open the valve
- Note that simultaneous inversions of inputs is not possible, so we put don't cares
  - ▹ Full & it becomes empty ab=11 $\rightarrow$ ab=01 $\rightarrow$ ab=00
  - ▹ Empty & it becomes full ab=00 $\rightarrow$ ab=01 $\rightarrow$ ab=11



				ab	
s^+	00	01	11	10	Z
1	–	2	**1**	?	0
2	3	**2**	1	?	0
3	**3**	4	–	?	1
4	?	?	?	?	?
s^-					

1. State table 3/3

- Now that the valve is open, the liquid will eventually reach `Sensor1`, so when ab=01 new state **4**
- Output should be maintained to '1' (valve is open) until ab=11, when the valve should be closed; we are in back to our initial situation **1**
- We are on the planet earth, `Sensor2` is **above** `Sensor1`, so situation in which there is liquid in the top & not in bottom is not possible → for ab=10 we always have don't cares



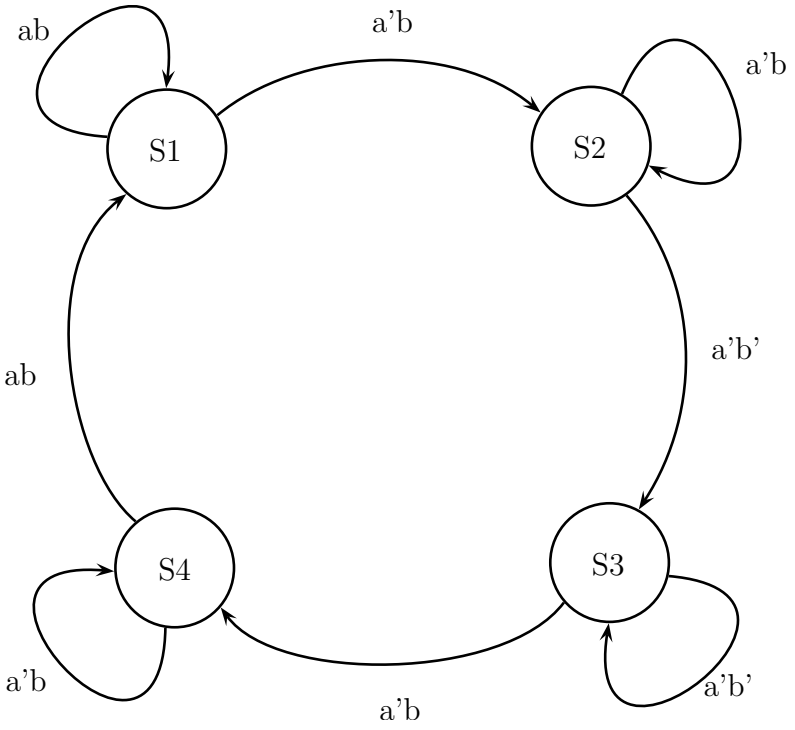
s^+ \ ab	00	01	11	10	Z
1	–	2	**1**	–	0
2	3	**2**	1	–	0
3	**3**	4	–	–	1
4	3	**4**	1	–	1

s^-

2. State graph

s^+ \ ab	00	01	11	10	Z
1	–	2	**1**	–	0
2	3	**2**	1	–	0
3	**3**	4	–	–	1
4	3	**4**	1	–	1

s^-



3. State encoding

We make the following (free for now!) choice:

$$1 \rightarrow \texttt{00};\ 2 \rightarrow \texttt{01};\ 3 \rightarrow \texttt{11};\ 4 \rightarrow \texttt{10}$$

	ab				
S^+	00	01	11	10	Z
1	–	2	**1**	–	0
2	3	**2**	1	–	0
3	**3**	4	–	–	1
4	3	**4**	1	–	1

S^-

	ab				
Y_2Y_1	00	01	11	10	Z
00	–	01	**00**	–	0
01	11	**01**	00	–	0
11	**11**	10	–	–	1
10	11	**10**	00	–	1

y_2y_1

4. K-Maps

Y_2Y_1 \ ab	00	01	11	10	Z
00	–	01	**00**	–	0
01	11	**01**	00	–	0
11	**11**	10	–	–	1
10	11	**10**	00	–	1

y_2y_1 \ ab	00	01	11	10
00	-	0	0	-
01	1	0	0	-
11	1	1	-	-
10	1	1	0	-

$$Y_2 = a'b' + y_2a'$$

y_2y_1 \ ab	00	01	11	10
00	-	1	0	-
01	1	1	0	-
11	1	0	-	-
10	1	0	0	-

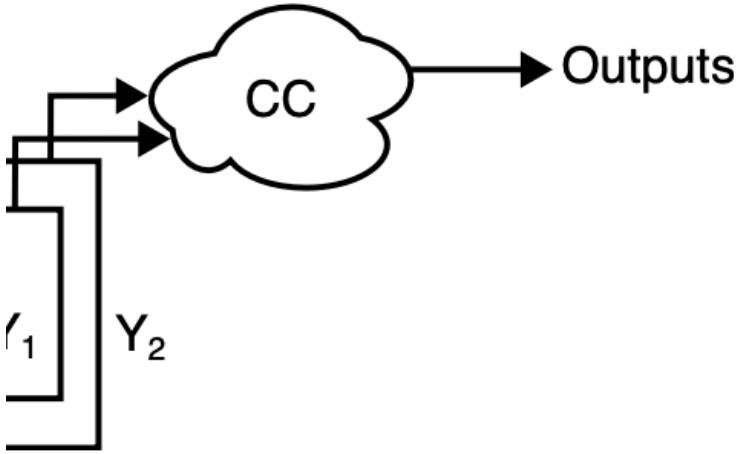
$$Y_1 = a'b' + y_2'a'$$

5. Outputs

- Outputs values are fixed for stable states; as a first (simple) method we will use state decoder to compute the output; more complex that will take transitions into account will be shown later
- State decoder – combinatorial circuit (CC) that uses current state variables only (Moore machine); combinatorial logic cloud (right fig.)
- Figure below can be misleading: we pick the inputs from the future state, not present; it should be present (drawing simplification)

	ab				
Y_2Y_1	00	01	11	10	Z
00	–	01	**00**	–	0
01	11	**01**	00	–	0
11	**11**	10	–	–	1
10	11	**10**	00	–	1

y_2y_1



$$Z = y_2y_1' + y_2y_1 = y_2(y_1 + y_1') = y_2$$

6. Enumerate all logic functions

- There are two logic functions (circuits) per state variable

$$Y_1 = a'b' + y_2'a'$$
$$Y_2 = a'b' + y_2a'$$

- One logic function for outputs:

$$Z = y_2$$

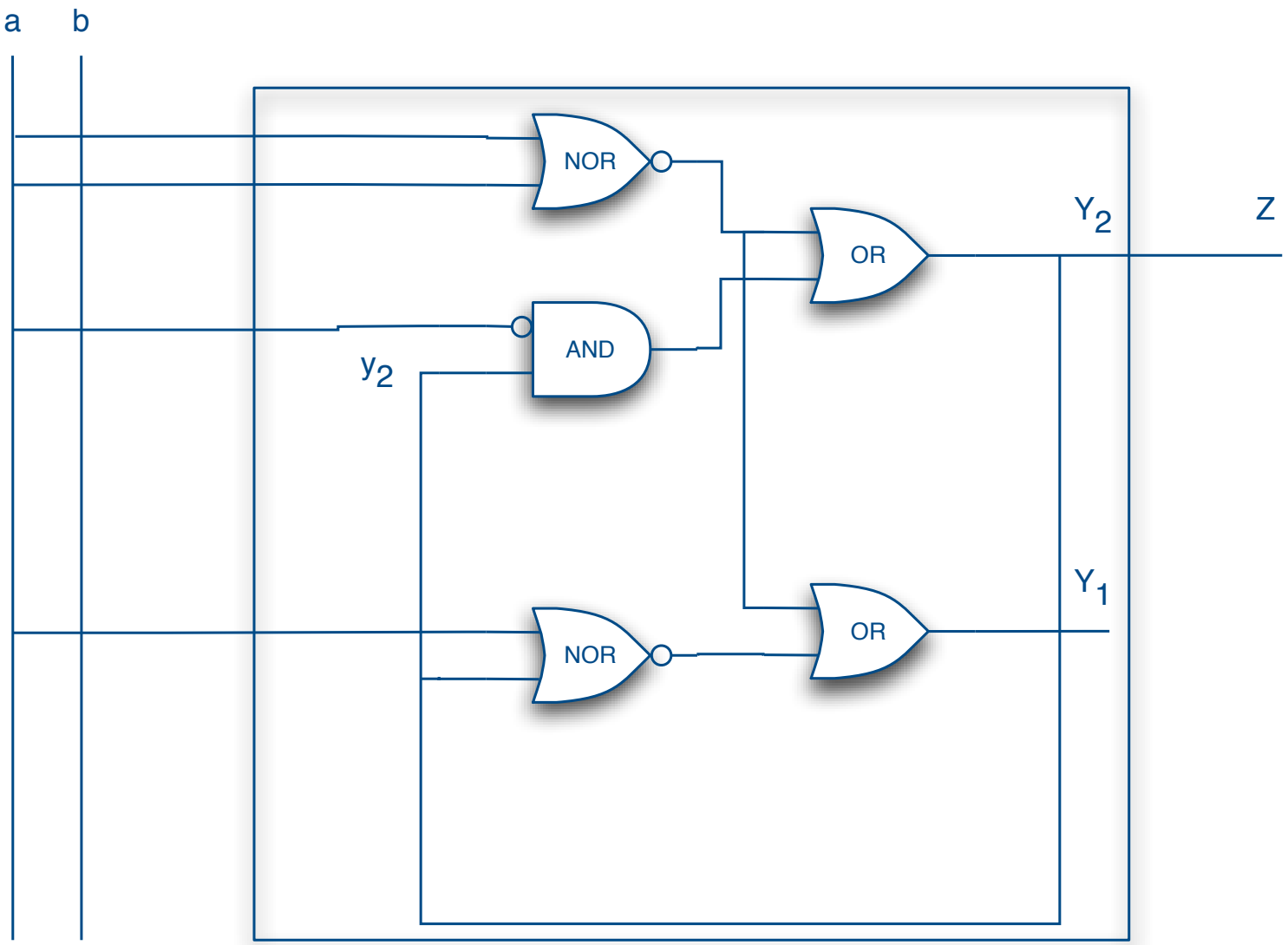
- Note only one feedback loop for the state variable, one for y_2 (due to optimisation); note the wire for the output (optimisation again)
- General form of expressions is:

$$\begin{aligned} Y_1 &= f_1(a, b, y_1, y_2) \\ Y_2 &= f_2(a, b, y_1, y_2) \\ Z &= f_3(y_1, y_2) \end{aligned}$$

- Certain terms could disappear (K-Map), but it is still a Moore machine; true difference with Mealy will be revealed later

7. Final logic circuit

$$Y_1 = a'b' + y_2'a' \qquad Y_2 = a'b' + y_2a' \qquad Z = y_2$$



4. Optimisation of primitive state tables

Optimisation & implementation

Sequential system synthesis → three steps:

- Establish primitive state table (Huffman Table)
  - Using verbal system specification (or state graphs for example) we first derive Primitive State Table (Huffman table)
  - Primitive State Table – one stable state per line of the table
  - We don't care about possible state redundancy, what we want is a fully functional state table that captures exactly what the system should do (i.e., this is a formal spec of our sequential system)
- State encoding
  - Chose unique binary code for each state
  - Free code choice
  - Pick the simplest one (for now) for K-maps
- Logic Equations
  - From encoded state table we derive logic expressions in their optimised form to allow optimal implementation

Sequential logic circuit complexity

- Final sequential logic circuit complexity will depend on the state table complexity, i.e. it's size in terms of number of states n
- For state encoding we use $log_2 n$ bits to encode n states, so more states, potentially mean more encoding bits
- And for each state bit we have:
  - One logic function (number of AND & OR gates)
  - One feedback loop (delay or memory element)
- By reducing the number of states in the system we can simplify the corresponding logic circuit: we save resources, which in turn means cheeper, faster, less power hungry circuit
- Similar to optimisation of the logic functions used in combinatorial systems, we want to reduce the number of states in state tables

Reducing the number of states

- Optimising the state table is possible thanks to the notion of state equivalence
- Two (stable) states are equivalent iff:
  1. they produce the **same output** (we are in the case of Moore Machine, output depends on the state only, so this one must be satisfied)
     AND
  2. for **all** combinations of inputs, all future states are the same or **equivalent** so that they have the same future
- Above can be extended to three and more states
  - ▹ All states should have same outputs and all pairs of states **two-by-two** are either the same or equivalent

Identical states and state fusion

- Identical states – 2 (or more) stable states have same output for same input combination (same column) & have the same future

	ab				
s^+	00	01	11	10	Z
1	**1**	2	3	5	0
6	**6**	2	3	5	0
s^-					

- Sate fusion – stable states for different input combinations, have the same future & same output

	ab				
s^+	00	01	11	10	Z
1	**1**	2	3	7	0
7	1	2	3	**7**	0
s^-					

State table after optimisation (fusion)

- Identical states

s^+ \ ab	00	01	11	10	Z
1	**1**	2	3	5	0
6	**6**	2	3	5	0

s^-

s^+ \ ab	00	01	11	10	Z
1	**1**	2	3	5	0

s^-

- Fused states

s^+ \ ab	00	01	11	10	Z
1	**1**	2	3	7	0
7	1	2	3	**7**	0

s^-

s^+ \ ab	00	01	11	10	Z
1	**1**	2	3	**1**	0

s^-

- In a given state table, finding all identical states and performing state fusion will yield state table optimisation

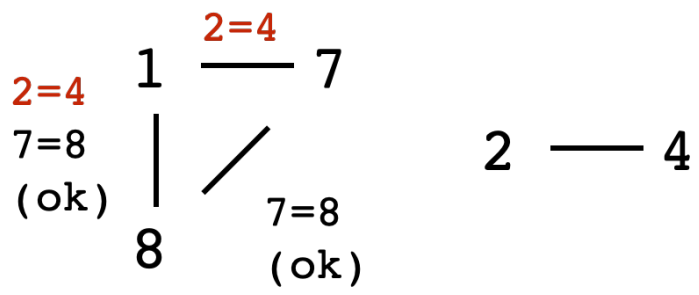
Fusing more than 2 states

- It is possible to fuse two or more states, if we can fuse **all pairs of states, two-by-two**
- It is possible to verify the above property with state fusion graphs that will show all fusion conditions that need to be checked before actual state fusion; take the example below:

ab

s^+	00	01	11	10	Z
1	**1**	2	8	7	0
7	1	4	8	**7**	0
8	1	4	**8**	8	0

s^-



- To be able to fuse 1,7 and 8: we need to fuse 7-8 and 2-4!
  - ▹ Table above is not complete, other states are not shown

Systematic search of equivalences

- Draw a table with $(n)(n-1)/2$ cells; where n is the number of states in the initial state table $\rightarrow$ equivalences conditions table
- We consider that each state is equivalent to itself – combination of n elements without repetitions
- Each table cell contains the condition of the potential equivalence between two stable states, and could have three options:
  - Not equivalent – we note this with and X; in the case of Moore machine any two pair of states with different outputs will be marked as such; 1st pass is always only about comparing the output value
  - Equivalent – we mark this OK; these two states can be fused without any conditions, but it doesn't necessarily mean that we will fuse them (this will depend on other opportunities and our goal $\rightarrow$ minimum number of states)
  - Conditional – some other pair of states need to be equivalent in order for these two to be equivalent

Example of equivalence conditions table

Example of equivalence conditions table for 10 states:

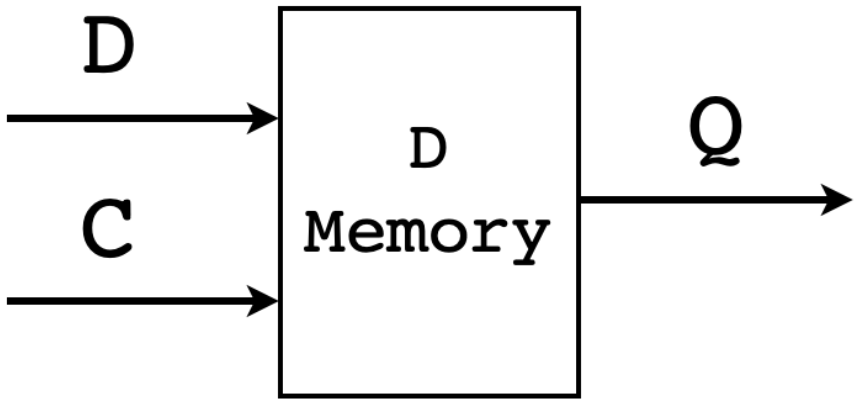
2									
3									
4									
5									
6									
7									
8									
9									
10									
	1	2	3	4	5	6	7	8	9

5. D Flip-Flop

Problem (verbal specification)

- Establish primitive state table, find optimised state table, write encoded state table & derive logic equations for D memory element or bi-stable, a class of circuits that can be one of two states '0' or '1'
  - These are basic memory elements, crucial sub-circuit in any digital system

- D-FF is a memory
  - Two inputs: D (data) & C (control)
  - One output Q
  - Output value Q is controlled by D and C



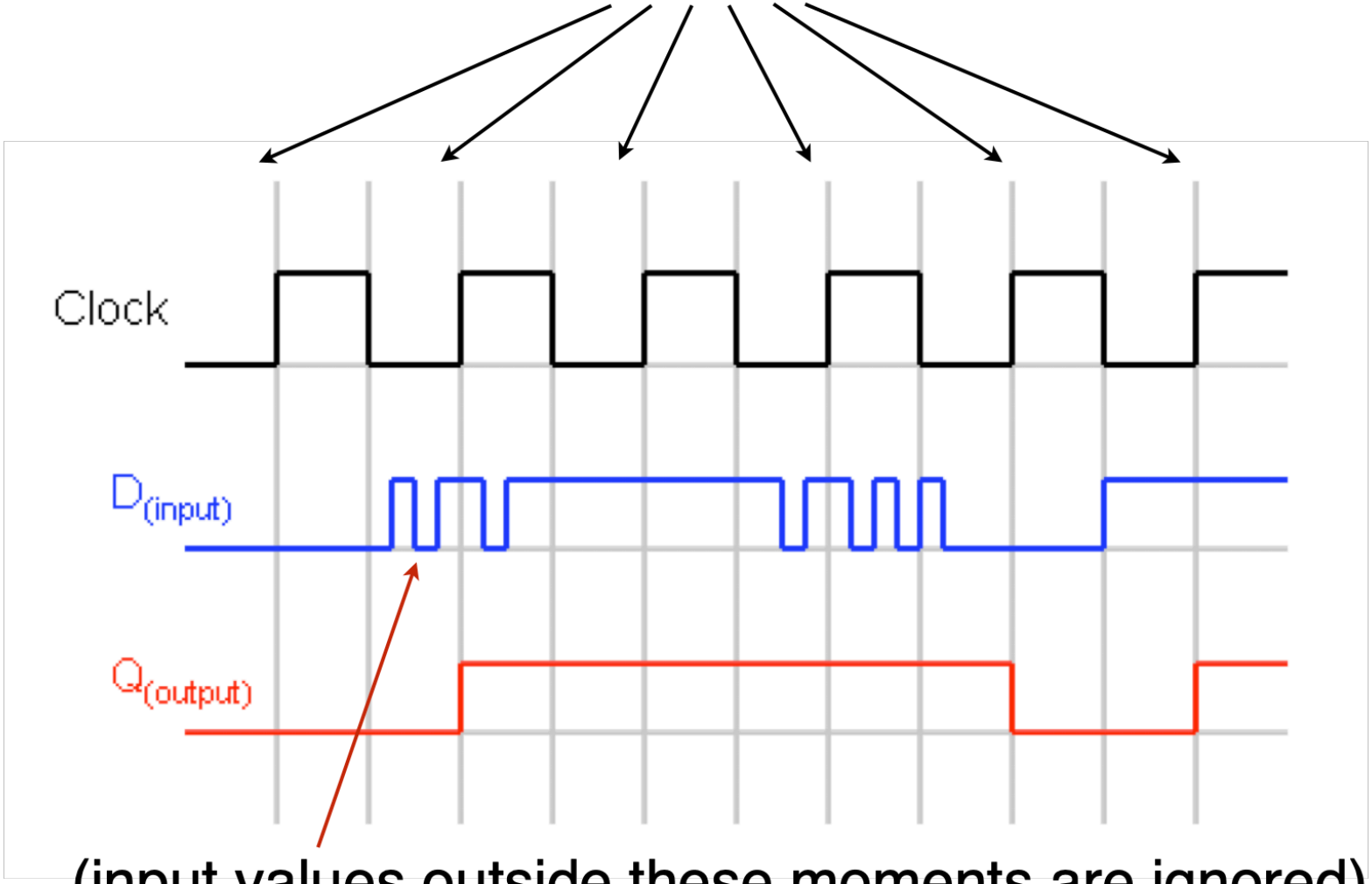
- Two variants:
  - Latches – sensitive to the level of C (we don't like these)
  - Flip-flops – sensitive to the transition of C (0 to 1 or inverse)

Specification of the edge triggered D-FF

- Assume signal C that controls the behaviour of D-FF; if C is periodic, we speak of clock (much more on this to come)
- Output Q takes value of D **only when rising edge occurs** on C
  - Rising edge – transition from '0' to '1'; we also speak of falling edge for transitions from '1' to '0'; for example DDR-DRAMs in your computers operate on both rising and falling edge
- In **between two rising edges of** C, the **old value of** D is maintained in the circuit; any variation on the input D is ignored
- If there is a simultaneous variation of C and D (rising edge of C and value of D switch from 0 to 1), Q is taking the old value of D, the one just before the rising edge
- In practice we must make sure that D arrives **before the rising edge** of C; if not we can't guarantee the content stored (metastability problem)

D Flip-Flop behaviour – timing diagram

Write to memory only at rising edge of a clock signal



(input values outside these moments are ignored)

D Flip-Flop – Primitive state table 1/5

- We start with an initial state, can be arbitrary – we can do that, but need to make sure that the system always "wakes up" in this state
- We call initial state **1** (bold), we assume already saved value of 0 (so Q=0) and input is CD is '00'
- As long as inputs don't change, system remains in **1**
- Assume now D is set to '1', before C is set to '1' (CD is 01) – we create new state 2: first transition and then the stable state (same column)
- This is a variation of D outside the rising edge of C → we ignore it, output is kept at 0

	CD				
S^+	00	01	11	10	Q
1	**1**	2			0
2	1	**2**			0
S^-					

D Flip-Flop – Primitive state table 2/5

- Imagine now that from state **1** input C switches from '0' to '1' while D remains constant set to '0'

- From initial state **1**, we see input transition CD from '00' to '10'
- We need new state, so we add state 3: transition and then the stable state (in the same column)
- State 3 describes the rising edge of C without change in D; there is no change on output
- We could do things the other way around and consider state 3 first rather than 2 (i.e. input change '10' first and then '01')

	CD				
S^+	00	01	11	10	Q
1	**1**	2		3	0
2	1	**2**			0
3				**3**	0
4					
S^-					

D Flip-Flop – Primitive state table 3/5

- Imagine now that from state **2** input C switches from '0' to '1' while D remains constant set to '1'

- From initial state **1**, we see input transition CD from '00' to '01', and then from '01' to '11'
- We need new state, so we add state 4: transition and then the stable state (in the same column)
- We have detected a valid input sequence needed to store '1'
- If you look at state 4 it says: Q is initially 0, then first D switched from '0' to '1', then C from '0' to '1' $\rightarrow$ output should be '1'

	CD				
S^+	00	01	11	10	Q
1	**1**	2		3	0
2	1	**2**	4		0
3				**3**	0
4			**4**		1
S^-					

D Flip-Flop – Primitive state table 4/5

- Imagine now that from state **1** we see *CD* that switches from '0' to '1' simultaneously, so *CD* goes from '00' to '11'

- New state 5: so once again transition & the stable state (in the same column of inputs)
- Spec says that in such case we should assume the value of *D* immediately **before** the simultaneous flip; *D* was '0' so *Q* remain at '0'
- State 5 tells us that there was simultaneous change of inputs, with 0 being sampled
- Same state 5 should be for the transition from 3 with *CD* is '11'

	CD				
S^+	00	01	11	10	*Q*
1	**1**	2	5	3	0
2	1	**2**	4		0
3			5	**3**	0
4			**4**		1
			5		0
S^-					

D Flip-Flop – Primitive state table 5/5

- Memorising, and keeping the memorised value of 1 in the state table
  - Assume initially CD=00 & Q=0 we are in **1**
  - CD input sequence is: 00, 01, 11 and the state sequence is: **1**,**2**,**4**
  - Output Q set to 1 and memorised
  - Then imagine C=0 we go to **7**
  - Whatever comes at input D, output Q will keep the value of 1 – succession of states **7**, **9**, **7**, **9**, in which D is ignored

	CD				
S^+	00	01	11	10	Q
1	**1**	2	5	3	0
2	1	**2**	4	8	0
3	6	2	5	**3**	0
4	9	7	**4**	8	1
5	6	2	**5**	3	0
6	**6**	2	5	3	0
7	9	**7**	4	8	1
8	9	7	4	**8**	1
9	**9**	7	5	10	1
10	6	2	5	**10**	0
S^-					

State equivalence table → 1st pass

- Primitive state table optimisation – this is a Moore machine, outputs must be the same for eventual state fusion (we put x)

2									
3	1-6								
4	x	x	x						
5	1-6		OK	x					
6	OK		OK	x	OK				
7	x	x	x	OK	x	x			
8	x	x	x	OK	x	x	OK		
9	x	x	x		x	x			
10			OK	x			x	x	x
	1	2	3	4	5	6	7	8	9

	CD				
	00	01	11	10	Q
1	**1**	2	5	3	0
2	1	**2**	4	8	0
3	6	2	5	**3**	0
4	9	7	**4**	8	1
5	6	2	**5**	3	0
6	**6**	2	5	3	0
7	9	**7**	4	8	1
8	9	7	4	**8**	1
9	**9**	7	5	10	1
10	6	2	5	**10**	0

State equivalence table → 2nd pass (conditions)

2	4-5 3-8								
3	1-6	1-6 4-5 3-8							
4	X	X	X						
5	1-6	1-6 4-5 3-8	OK	X					
6	OK	1-6 4-5 3-8	OK	X	OK				
7	X	X	X	OK	X	X			
8	X	X	X	OK	X	X	OK		
9	X	X	X	4-5 8-10	X	X	4-5 8-10	4-5 8-10	
10	1-6 3-10	1-6 4-5 3-10	OK	X	3-10	3-10	X	X	X
	1	2	3	4	5	6	7	8	9

	CD				
	00	01	11	10	*Q*
1	**1**	2	5	3	0
2	1	**2**	4	8	0
3	6	2	5	**3**	0
4	9	7	**4**	8	1
5	6	2	**5**	3	0
6	**6**	2	5	3	0
7	9	**7**	4	8	1
8	9	7	4	**8**	1
9	**9**	7	5	10	1
10	6	2	5	**10**	0

State equivalence table → 3rd pass (conditions)

2	4-5 3-8	first condition 4-5 not OK							
3	1-6	1-6 4-5 3-8							
4	X	X	X						
5	1-6	1-6 4-5 3-8	OK	X					
6	OK	1-6 4-5 3-8	OK	X	OK				
7	X	X	X	OK	X	X			
8	X	X	X	OK	X	X	OK		
9	X	X	X	4-5 8-10	X	X	4-5 8-10	4-5 8-10	
10	1-6 3-10	1-6 4-5 3-10	OK	X	3-10	3-10	X	X	X
	1	2	3	4	5	6	7	8	9

	CD				
	00	01	11	10	*Q*
1	**1**	2	5	3	0
2	1	**2**	4	8	0
3	6	2	5	**3**	0
4	9	7	**4**	8	1
5	6	2	**5**	3	0
6	**6**	2	5	3	0
7	9	**7**	4	8	1
8	9	7	4	**8**	1
9	**9**	7	5	10	1
10	6	2	5	**10**	0

Equivalent state table

- Equivalent states: same outputs, same future! – states 1 & 6 are equivalent → 1; and also states 3 & 10 → 3

S^+ \ CD	00	01	11	10	Q
1	**1**	2	5	3	0
2	1	**2**	4	8	0
3	6	2	5	**3**	0
4	9	7	**4**	8	1
5	6	2	**5**	3	0
6	**6**	2	5	3	0
7	9	**7**	4	8	1
8	9	7	4	**8**	1
9	**9**	7	5	10	1
10	6	2	5	**10**	0

S^-

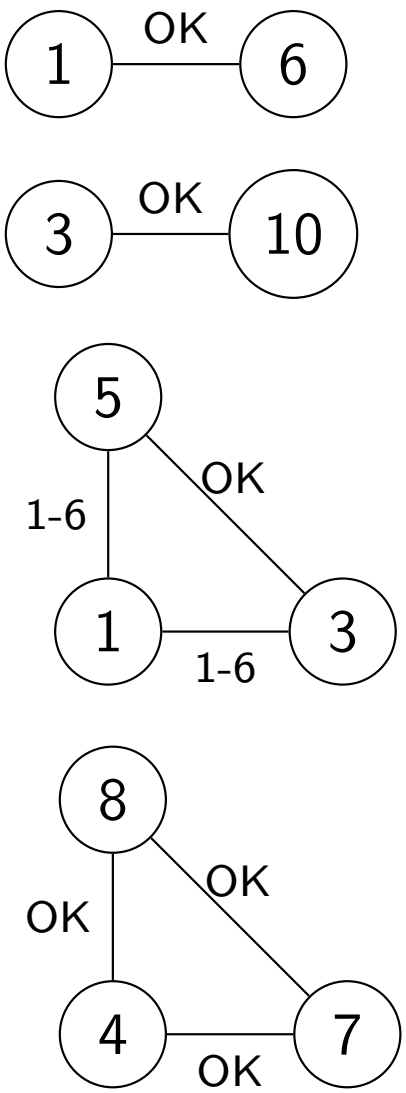
S^+ \ CD	00	01	11	10	Q
1	**1**	2	5	3	0
2	1	**2**	4	8	0
3	1	2	5	**3**	0
4	9	7	**4**	8	1
5	1	2	**5**	3	0
7	9	**7**	4	8	1
8	9	7	4	**8**	1
9	**9**	7	5	3	1

S^-

State fusion graphs

2	4-5 3-8								
3	1-6	1-6 4-5 3-8							
4	X	X	X						
5	1-6	1-6 4-5 3-8	OK	X					
6	OK	1-6 4-5 3-8	OK	X	OK				
7	X	X	X	OK	X	X			
8	X	X	X	OK	X	X	OK		
9	X	X	X	4-5 8-10	X	X	4-5 8-10	4-5 8-10	
10	1-6 3-10	1-6 4-5 3-10	OK	X	3-10	3-10	X	X	X
	1	2	3	4	5	6	7	8	9

first condition 4-5 not OK



Final optimised table

- 1, 3, 5 → 1
- 4, 7, 8 → 4
- 2 → 4 and 9 → 9

S^+ \ CD	00	01	11	10	Q
1	**1**	2	5	3	0
2	1	**2**	4	8	0
3	1	2	5	**3**	0
4	9	7	**4**	8	1
5	1	2	**5**	3	0
7	9	**7**	4	8	1
8	9	7	4	**8**	1
9	**9**	7	5	3	1

S^-

S^+ \ CD	00	01	11	10	Q
1	**1**	2	**1**	**1**	0
2	1	**2**	4	4	0
4	9	**4**	**4**	**4**	1
9	**9**	4	1	1	1

S^-

Final optimised table, re-encoded (not mandatory)

S^+ \ CD	00	01	11	10	Q
1	**1**	2	**1**	**1**	0
2	1	**2**	4	4	0
4	9	**4**	**4**	**4**	1
9	**9**	4	1	1	1

S^-

S^+ \ CD	00	01	11	10	Q
1	**1**	2	**1**	**1**	0
2	1	**2**	3	3	0
3	4	**3**	**3**	**3**	1
4	**4**	3	1	1	1

S^-

Optimisation: we moved from 10 to 4 states, meaning 2 state variables instead of 4. This is a significantly less resources to do exactly the same thing!

Binary encoded optimised state table

We (still!) chose codes arbitrarily, so:

$$1 \rightarrow \texttt{00},\ 2 \rightarrow \texttt{01},\ 3 \rightarrow \texttt{11},\ 4 \rightarrow \texttt{10}$$

S^+ \ CD	00	01	11	10	Q
1	**1**	2	**1**	**1**	0
2	1	**2**	3	3	0
3	4	**3**	**3**	**3**	1
4	**4**	3	1	1	1

S^-

Y_2Y_1 \ CD	00	01	11	10	Q
00	**00**	01	**00**	**00**	0
01	00	**01**	11	11	0
11	10	**11**	**11**	**11**	1
10	**10**	11	00	00	1

y_2y_1

We have two K-Maps on the right for two functions!

Binary encoded optimised state table

Y_2Y_1 \ CD	00	01	11	10	Q
00	**00**	01	**00**	**00**	0
01	00	**01**	11	11	0
11	10	**11**	**11**	**11**	1
10	**10**	11	00	00	1

y_2y_1

y_2y_1 \ ab	00	01	11	10
00	0	0	0	0
01	0	0	1	1
11	1	1	1	1
10	1	1	0	0

$$Y_2 = y_1C' + y_2C$$

y_2y_1 \ ab	00	01	11	10
00	0	1	0	0
01	0	1	1	1
11	0	1	1	1
10	0	1	0	0

$$Y_1 = C'D + y_1C$$

Output

	CD				
Y_2Y_1	00	01	11	10	Q
00	**00**	01	**00**	**00**	0
01	00	**01**	11	11	0
11	10	**11**	**11**	**11**	1
10	**10**	11	00	00	1
y_2y_1					

$$Z = y_2y_1 + y_2y_1' = y_2(y_1' + y_1) = y_2$$